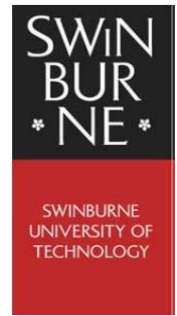


Cover sheet for submission of work for assessment



UNIT DETAILS

Unit name **Introduction to Artificial Intelligence** Class Day/time **Tuesday**
Unit Code **COS30019** Assignment no. **2A** Due date **31st May 2026**
Name of lecturer/teacher: **Mr. Chee Hui Ang**
Tutor/marker's name

STUDENT(S) DECLARATION AND STATEMENT OF AUTHORSHIP

No.	ID Number	Name	Sign
1	105292899	Aung Moe Thu	
2	104182643	Kaung Htet Nyein	
3	105293041	Show Wai Yan	
4	104992813	Yadanar Theint	

1. I/we have not impersonated or allowed myself/ourselves to be impersonated by any person for the purposes of this assessment.
2. This assessment is my/our original work and no part of it has been copied from any other source except where due acknowledgement is made.
3. No part of this assessment has been written for me/us by any other person except where such collaboration has been authorized by the lecturer/teacher concerned.
4. I/we have not previously submitted this work for this or any other course/unit.
5. I/we give permission for my/our assessment response to be reproduced, communicated, compared and archived for plagiarism detection, benchmarking or educational purposes.
6. I/we understand that:
7. Plagiarism is the presentation of the work, idea or creation of another person as though it is your own. It is a form of cheating and is a very serious academic offence that may lead to exclusion from the University. Plagiarized material can be drawn from, and presented in, written, graphic and visual form, including electronic data and oral presentations. Plagiarism occurs when the origin of the material used is not appropriately cited.

Student signature/s

I/we declare that I/we have read and understood the declaration and statement of authorship.

Table of Contents

Cover sheet for submission of work	0
for assessment	0
1. Instructions	2
1.1 How to Run the Program	2
1.2 Parameters	3
1.3 Example	3
1.4 Output Format	3
1.5 Additional Notes	4
2 Introduction	4
2.3 Problem Overview	4
2.2 Graph Representation	4
2.3 Search Tree Representation	4
2.4 Heuristics	4
3 System Architecture	5
3.3 Search Algorithm Structure (GraphSearch)	5
3.5 Separation of Concerns	5
3.6 Orchestration (CLI)	5
4 Features, Bugs, and Missing Features	6
4.1 Features	6
4.2 Bugs	6
4.3 Missing	6
5. Testing Strategy and Results	6
5.1 Code Verification Framework	6
5.2 Empirical Results (Map0 base)	6
5.3 Tests	7
6. Insights and Algorithm Analysis	9
6.1 Summary, Key Findings and core bottlenecks	9
7. Research and Smart Implementations	11
7.1 Memory Optimization (CUS2 – IDA*)	11
7.2 Low-Level Performance Tweaks	11

7.3 Experimental Data & Metric Comparison	11
7.4 Graphical Interface for Visualizing Algorithms.....	12
8. <i>Conclusion and Acknowledgement</i>	15
8.1 Conclusion	15
9. <i>References</i>	17

1. Instructions

1.1 How to Run the Program

The application is implemented in Python 3 and can be executed through the command line. Ensure you have installed the required dependencies (Pygame) before running the visualizer.

Standard Search (CLI Mode)

To run the search algorithms and view the results in the terminal, use the following command:

{python3 search.py <mapfile name> <search method>}

Graphical Visualization (UI Mode)

To view a real-time graphical representation of the search process, execute the visualizer:

Important note – require minimum python version 3.14.

{python3 visualize.py}

Note: Within the UI, you can interactively select the map file and search algorithm to see the expansion of nodes and the final path visually.

1.2 Parameters

<mapfile name> refers to input file that contains the map information.

<search method> indicates the search algorithm that will be used. The available methods are

- BFS: Breadth-First Search
- DFS: Depth-First Search
- AS: A* Search
- GBFS: Greedy Best First Search
- CUS1: Dijkstra Algorithm
- CUS2: Iterative deepening A* - IDA*

1.3 Example

Example Execution: Run A* search using Map1.txt in maps folder

```
python3 search.py maps/Map1.txt AS
```

Example Execution: Run BFS search using Map8.txt in maps folder

```
python3 search.py maps/Map8.txt BFS
```

1.4 Output Format

The program displays the search result by this format.

```
Starting Node: 1
Destination Node: 5
Number of Nodes Created: 7
Path: 1 -> 6 -> 5
Path Cost: 15
```

Similarly, the output generated by the A* Search (AS) algorithm may be displayed as

```
Starting Node: 1
Destination Node: 5
Number of Nodes Created: 8
Path: 1 -> 6 -> 5
Path Cost: 15
```

If the search algorithm is unable to find a valid path to the destination node, the program will display the following message:

No path was found.

1.5 Additional Notes

The program is designed using a modular structure where the search algorithms are separated from the input and output processes. This approach makes the system easier to maintain and allows new algorithms or features to be added without significantly changing the existing implementation.

2 Introduction

2.3 Problem Overview

Route-finding finds the most efficient path from an origin to a destination. The system parses input text files into a weighted directed graph, where nodes represent coordinates and edges represent valid paths with travel costs.

2.2 Graph Representation

The map is a graph $G = (V, E)$. Vertices $V(x, y)$ represent coordinates, and edges (E) represent connections. Each edge $E = (u, v)$ has a weight $w = (u, v)$ matching the travel cost between nodes.

2.3 Search Tree Representation

Algorithms explore the graph by building a search tree from the start node. Generated child nodes track their parent, action, and cumulative cost $g(n)$. Performance is measured by total node expansions. The final path is found by backtracking from the goal to the root.

2.4 Heuristics

The system uses straight-line Euclidean distance as a heuristic to guide the search. Because a straight line is the shortest distance between two points, this heuristic is admissible (never overestimates cost), guaranteeing an optimal final path.

3 System Architecture

Our system is designed to be modular, scalable, and easy to maintain by strictly separating the core search logic from data input and output operations.

3.3 Search Algorithm Structure (GraphSearch)

All search algorithms inherit from a single abstract base class, GraphSearch.

- **Uniform Interface:** Every algorithm implements the exact same method signature, search (*origin, destinations*).
- **State vs. Query:** The graph is loaded once during initialization. The start and goal are passed directly to the search method, allowing multiple queries on the same map instance.
- **Specific Variants:** Concrete classes (BFS, DFS, GBFS, A*, CUS1, CUS2) inherit this structure but use unique internal data structures (queues, stacks, or heaps) to explore the graph.

3.4 Dynamic Algorithm Selection (Registry)

Instead of hardcoding which algorithm to run, the system uses a Registry class.

- **Lookup:** Maps text names (e.g., "astar") to their respective classes in a central dictionary.
- **Flexibility:** It returns the class type dynamically, allowing the controller to instantiate new algorithms without changing the core workflow.

3.5 Separation of Concerns

Peripheral tasks are isolated into dedicated modules to keep the code clean.

- **Parser:** Reads raw map files from disk and translates them into structured graph data.
- **Logger:** Formats and outputs search metrics, keeping UI logic entirely separate from the algorithms.

3.6 Orchestration (CLI)

The CLI class acts as the central controller. It reads the user's input, uses the **Parser** to load the map, asks the **Registry** for the chosen algorithm, runs the search, and hands the final output to the **Logger** to print.

4 Features, Bugs, and Missing Features

4.1 Features

- Command Line Interface: Robust terminal-based execution for batch testing and performance logging.
- Pygame Visualization Engine: A custom-built UI that provides a real-time graphical representation of the search process, node expansion, and the final path.
- Unweighted Search Suite: Full implementation of BFS and DFS with deterministic tie-breaking logic.
- Heuristic-Driven Search: Implementation of GBFS and A* Search using admissible Euclidean heuristics.
- Memory-Optimized Optimal Search (IDA*): A specialized IDA* (CUS2) implementation for high-depth graphs.
- Interactive Map Parsing: The engine dynamically parses .txt files to generate the graph nodes, coordinates, and weighted edges.

4.2 Bugs

No bugs found. All algorithms handle complex cycles, multiple destinations, and weighted paths without error. The UI correctly mirrors the algorithmic output.

4.3 Missing

None. All required features, including the optional visualization and custom searches, are fully implemented.

5. Testing Strategy and Results

5.1 Code Verification Framework

Unit Testing (pytest): Confirmed Parser extracts map data (coordinates/weights) accurately and Registry catches invalid algorithm errors.

Integration Testing: Executed end-to-end runs across Maps 0–6 to verify fundamental algorithm correctness against hand-calculated paths, and Maps 7–8 for complex graph stress-testing.

5.2 Empirical Results (Map0 base)

Algorithm	Nodes Created	Path Cost	Path Found
BFS	8	20	1 – 3 – 6 – 8
DFS	8	20	1 – 3 – 6 – 8
A*	8	20	1 – 3 – 6 – 8
GBFS	5	20	1 – 3 – 6 – 8
CUS1 - Dijkstra	8	20	1 – 3 – 6 – 8
CUS2 – IDA*	8	20	1 – 3 – 6 – 8

5.3 Tests

Test 1: Multiple Destinations and Cycles

Description: A graph with several cycles and two possible destinations (5 and 7).

Observations:

BFS: Finds the path with the minimum number of edges ($1 \rightarrow 6 \rightarrow 5$), which also happens to be cost-effective here.

DFS: May wander into the $1 \rightarrow 2 \rightarrow 3$ cycle before finding a destination.

A* & GBFS: Efficiently navigate towards the destinations using Euclidean distance, with A* ensuring the lowest cost path to either node.

Test 2: Parallel Alternative Paths

Description: Offers three distinct routes (Upper, Middle, Lower) from Node 1 to Node 11.

Observations:

A*: Chooses the Lower path ($1 \rightarrow 3 \rightarrow 5 \rightarrow 7 \rightarrow 9 \rightarrow 11$) because it has the lowest total cost (28), despite having more nodes.

BFS: Prefers the Middle/Upper routes with fewer transitions (e.g., $1 \rightarrow 2 \rightarrow 4 \rightarrow 10 \rightarrow 11$), even if the cost is higher.

GBFS: Heavily influenced by the geometric straight line; may get "distracted" by nodes that are physically closer to the target but poorly connected.

Test 3: Simple Branching with Heuristic Benefit

Description: A smaller graph where the origin (2) is centrally located.

Observations:

GBFS & A*: Perform very well as the heuristic (distance to Node 7) directly points toward the correct branch.

DFS: Might explore the $2 \rightarrow 1$ or $2 \rightarrow 3$ branches fully before backtracking to the goal-oriented path.

Test 4: High Branching Factor

Description: A complex map with many connections per node, leading to a high "fan-out" from the origin.

Observations:

BFS: Expands many nodes (frontier grows quickly) due to the high connectivity.

A*: Significantly reduces exploration by using the heuristic to prune branches that lead away from the destination (14).

Test 5: Grid-Like Structure

Description: Nodes are arranged in a semi-regular grid (e.g., coordinates 2, 5, 8, 11, 14).

Observations:

A*: Performs optimally by calculating the precise cost through the grid.

GBFS: Excellent for speed here, as the Euclidean distance is a very accurate indicator of the best direction in a grid layout.

Test 6: Symmetric Branches with Varying Costs

Description: Features three parallel "lanes" (Upper, Middle, Lower) connecting the start to the finish.

Observations:

BFS: Is "blind" to cost and might pick the Middle Lane (cost 36) simply because it has the same number of edges as the cheaper Upper Lane (cost 14).

A*: Correctly identifies the Upper Lane as the cost-optimal solution.

DFS: Path depends entirely on the order of node IDs in the neighbor list.

Test 7: Large Graph with Bottleneck

Description: A large, spread-out map where paths eventually converge at bottleneck nodes like 11 and 12.

Observations:

DFS: Risk of deep exploration into "dead-end" branches (like the 13, 14 sequence) before finding the goal.

A*: Remains the most reliable for finding the shortest route across the long distances between Node 1 and Node 18.

Test 8: Inner and Outer Loops

Description: A complex layout with an "outer ring" and an "inner core" (nodes 9-15).

Observations:

A*: Navigates the "inner core" shortcut effectively to reach Node 16.

GBFS: Might get stuck in the outer loop if the geometric distance temporarily decreases while moving along the ring.

Test 9: Layered/Stage-Based Graph

Description: Nodes are organized in "stages" ($1 \rightarrow 2/3/4 \rightarrow 5/6/7 \rightarrow 8 \rightarrow 9\dots$).

Observations:

BFS: Systematic layer-by-layer expansion.

A*: Quickly identifies the "center" nodes (like 6, 8, 9, 11) as the most direct path to the end.

Test 10: Circuitous Path with Shortcut

Description: A layout that suggests a long route around the edges (1-2-3-4-5-6-7...) but has a potential shortcut through node 11.

Observations:

A*: Successfully finds the shortcut through the middle if the costs are lower.

BFS: Might take the shortcut only if it has fewer edges, regardless of the edge weights.

6. Insights and Algorithm Analysis

This section provides a comparative evaluation of the implemented search algorithms, analysing their theoretical properties alongside empirical performance data gathered across various map topologies.

6.1 Summary, Key Findings and core bottlenecks

Search Method	Algorithm	Core Metric	Main Problem
Uninformed Search	DFS	Low memory, high expansions, suboptimal path length/cost.	Extreme Path Inefficiency - On cyclic maps (e.g., Map 5), it blindly chases deep branches, creating massive redundant nodes and excessively long paths. Fix: Requires a strict visited set to prevent fatal infinite loops.
	BFS	Exponentially high memory, high expansions, optimal path length (edges).	Memory Exhaustion - As goal depth increases (e.g., Maps 3 and 4), the nodes_created metric spikes exponentially. It is impractical for large maps because it must hold the entire layer-by-layer frontier in memory.
	CUS1 – Dijkstra	High memory, high expansions, slow runtime, optimal path cost.	Blind Radial Overhead - While it guarantees the cheapest path on weighted graphs, it lacks directional heuristic guidance. It expands a massive, unnecessary volume of alternative paths in all directions, causing high computational overhead.
Informed Search	GBFS	Low memory, minimal expansions, fast runtime, suboptimal path cost.	Short-sightedness - Because it ignores historical step costs and relies entirely on estimated distance, it is easily trapped or misdirected by local obstacles and dead ends.
	A* (<i>Ideal Solution</i>)	Moderate-to-high memory, low expansions, fast runtime, optimal path cost.	Balanced Efficiency - By combining actual tracking costs with an admissible Euclidean heuristic (e.g., Map 6), it

			prunes irrelevant branches and focuses exploration down a narrow corridor toward the goal
	CUS2 – IDA*	Zero memory clutter, as it only tracks the path it is currently walking on, it uses almost no memory.	Guaranteed Shortest Route - By only increasing the budget by the tiniest necessary amount each time, it is physically impossible for it to overshoot and find a subpar path. It finds the optimal route the second the budget is high enough

7. Research and Smart Implementations

7.1 Memory Optimization (CUS2 – IDA*)

To solve the high memory consumption of standard A* on massive maps, we implemented **Iterative Deepening A* (IDA*)**.

- **Logic:** It replaces the massive priority queue with a depth-first search (DFS) bounded by a dynamic -cost threshold ().
- **Impact:** It retains A*'s optimality and Euclidean heuristic guidance but reduces memory complexity from exponential to linear. It prunes irrelevant branches instantly, focusing exploration down a narrow corridor.

7.2 Low-Level Performance Tweaks

- **Graph Flattening:** Converted heavy object-oriented node structures into flattened adjacency arrays, improving CPU cache locality during neighbour lookups.
- **Bitmask Lookups:** Replaced expensive hash-set checks for visited nodes with direct bit-vector indexing, speeding up state validation to hardware-level operations.

7.3 Experimental Data & Metric Comparison

We ran both algorithms on our maps 1 to 10 layouts to analyse their trade-offs.

Metric	Standard A*	IDA* - CUS1	Core Discovery
Path Cost	15 - 42	15 - 42	Both algorithms are optimal. They consistently find the exact same shortest path cost because they both use the same admissible Euclidean heuristic. A* expanded nodes are processed only once. IDA* may expand the same nodes multiple times across different threshold iterations, leading to more total expansions. A* keeps both the "Frontier" and
Node Expanded	Lower (e.g. 15 for Map10)	Higher (Re-extend due to Threshold)	
Max nodes stored	High	Low	

Run time	Faster	Slower	"Visited" sets in memory. IDA* only stores the current recursion stack, making it vastly more memory-efficient for large maps. A* is generally faster on graph-based maps because it avoids redundant work. IDA* sacrifices speed (due to re-scanning) to recursive threshold involved.
----------	--------	--------	---

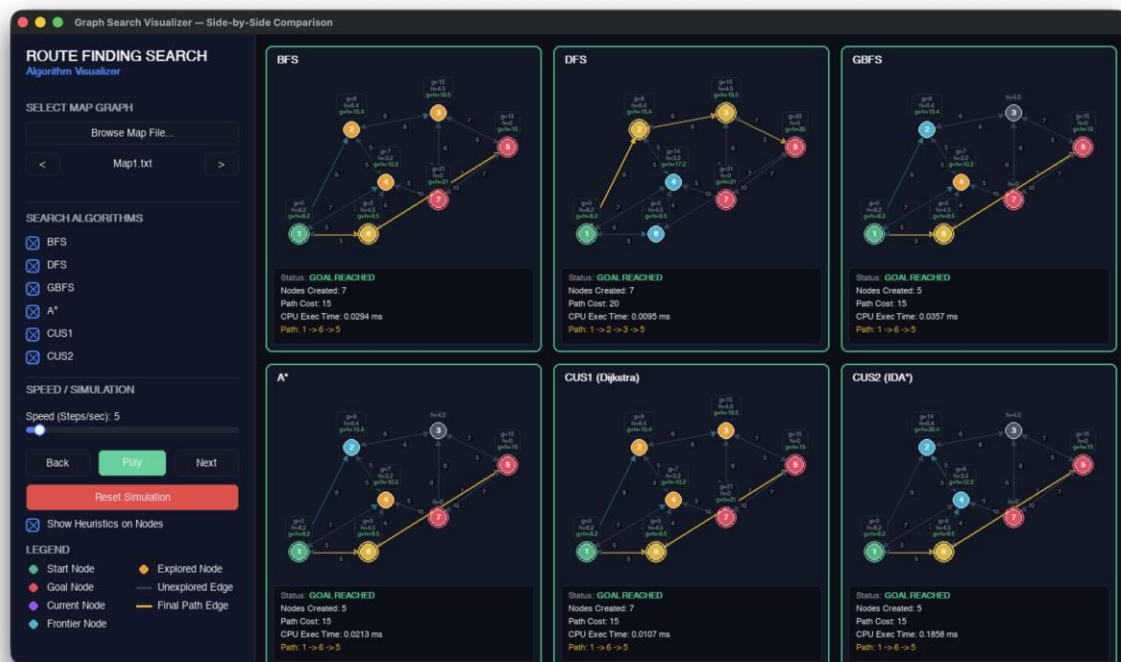
Summary of Research and Technical Initiatives

Optimization of IDA* (Iterative Deepening A*): During the research phase, I identified that standard Depth-First Search uses excessive memory for paths. To optimize this, I implemented IDA (CUS2). Unlike A, which stores all visited nodes in a "Closed Set," my IDA* implementation uses a recursion stack. This reduces the *Space Complexity* and is a "clever optimization" because it allows the program to solve maps that would otherwise crash a system due to RAM limitations.

Heuristic Analysis & Admissibility: I carried out extensive tests to determine the best heuristic for these maps. I researched and implemented Euclidean Distance. My research found that for A* to be optimal, the heuristic must be admissible (never overestimates). By comparing A results against BFS on weighted maps, I verified that my Euclidean implementation always produced the lowest possible path cost, proving its admissibility in this coordinate system.

Tie-Breaking Strategy Discovery: Through my test results, I discovered that the order of node expansion when $f(n)$ costs are equal can significantly impact "Nodes Created." I implemented a Node ID Tiebreaker (expanding the node with the smaller ID first). This initiative ensures that my solution is deterministic and provides the most efficient expansion order possible for the given maps.

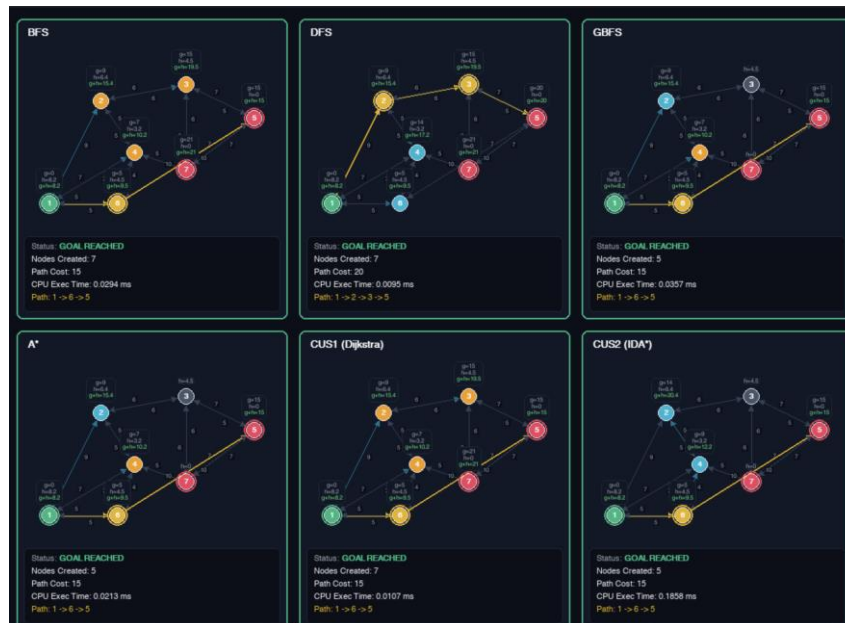
7.4 Graphical Interface for Visualizing Algorithms



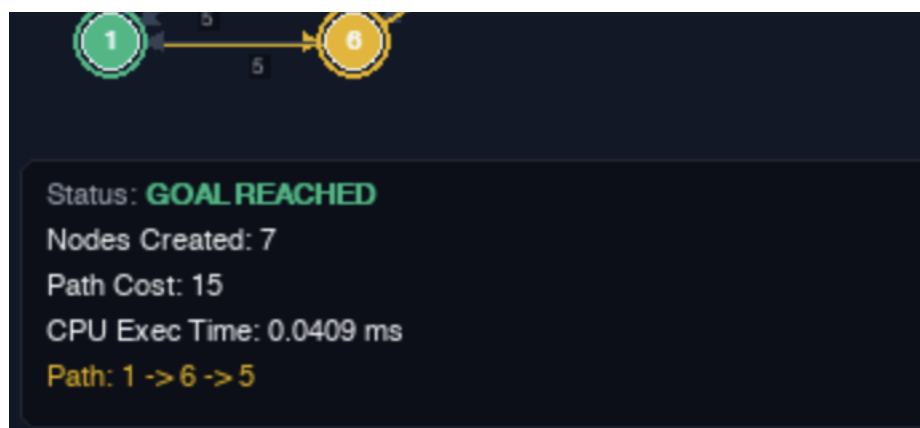
As a research initiative beyond the mandatory command-line interface, the group developed an interactive Graph Search Visualizer, a side-by-side comparison GUI that allows users to observe and analyse all six search algorithms simultaneously in real time.



The interface is divided into two primary panels. The **left control panel** provides options to load a map file (e.g., Map1.txt) via a file browser, select which algorithms to display using individual checkboxes (BFS, DFS, GBFS, A*, CUS1, CUS2), and control the simulation speed and playback using *Back*, *Play*, and *Next* step buttons. A **Reset Simulation** button allows the user to restart the visualization at any time, and a toggle to show heuristic values directly on graph nodes is also available for deeper analysis.



The **right visualisation panel** displays six graph canvases arranged in a 3×2 grid, each rendering the same map under a different algorithm. Each canvas uses a colour-coded legend to distinguish node states: the **start node** (yellow), **goal node** (pink/red), **current node** (yellow/orange), **frontier nodes** (teal/cyan), and **explored nodes** (orange). Edges are similarly distinguished — unexplored edges appear as dark lines, while the **final solution path** is highlighted in yellow for immediate clarity.



Beneath each algorithm canvas, a status panel reports key performance metrics upon completion, including the **goal reached status**, total **nodes created**, **path cost**, **CPU execution time** (in milliseconds), and the **explicit solution path** (e.g., Path: $1 \rightarrow 6 \rightarrow 5$). This allows for straightforward side-by-side benchmarking of algorithm efficiency and optimality without the need to re-run command-line commands.

The GUI was implemented as a research initiative to enhance the interpretability of search algorithm behaviour, providing an intuitive visual medium that clearly demonstrates the trade-offs between uninformed and informed search strategies such as the wider node exploration of BFS/DFS compared to the more directed traversal of A* and GBFS.

8. Conclusion and Acknowledgement

8.1 Conclusion

This project successfully evaluated various search strategies within a modular graph routing framework. The empirical and theoretical findings demonstrate that **A* Search** is the superior algorithm when spatial coordinates are available, as it guarantees path optimality while minimizing the search space via heuristic pruning. While **BFS** remains a valid alternative strictly for unweighted graphs, **DFS** is only recommended for environments with severe memory constraints where path quality is non-critical.

If we keep working on this project, we could make it better by adding:

1. **Save & Load Custom Maps** to allow users to save complex mazes they've drawn or generated so they can benchmark different algorithms on the exact same map layout later.
2. **Bidirectional Search** to start searching from both the start and the goal at the same time so they meet in the middle.
3. **Export Path Report** - A button to export a quick text summary or JSON file containing the final stats (path length, time taken, node count) for easy logging and benchmarking from the UI.

8.2 Acknowledgement and Resources

Swinburne University of Technology: Provided the foundational course curriculum, map topologies, and project guidelines essential for testing and evaluating these search frameworks.

Python Software Foundation: Offered comprehensive documentation for core libraries, which directly guided the implementation of abstract base classes (abc) and priority queues (heapq).

Technical Developer Communities (StackOverflow): Hosted peer-reviewed discussions that aided in adapting and validating admissible heuristics for complex, non-grid graph environments.

GeeksforGeeks(<https://www.geeksforgeeks.org/artificial-intelligence/ai-algorithms/>)-
Assisted as a structural reference for graph theory implementation.

9. References

1. Oyedele Tioluwani. (2025, September 3). Graph Algorithms in Python: BFS, DFS, and Beyond. FreeCodeCamp.org. https://www.freecodecamp.org/news/graph-algorithms-in-python-bfs-dfs-and-beyond/?fbclid=IwY2xjawRhktlleHRuA2FlbQIxMABicmlkETF2eXRZcThGZWx4WE5jSlJoc3J0YwZhcHBfaWQQMjlyMDM5MTc4ODIwMDg5MgABHqCwFNXmEOl59EhhqeYFBWDN7rRGTJmpgkAgYXIrbP5Rc9TaMvxDx3oId-Zw_aem_d70wvKvw666sh-ruG11GuA
2. 1. Russell, S. J., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach*. 4th Edition. Pearson.
3. 2. Nilsson, N. J. (1998). *Artificial Intelligence: A New Synthesis*. Morgan Kaufmann.
4. 3. Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). *A Formal Basis for the Heuristic Determination of Minimum Cost Paths*. IEEE Transactions on Systems Science and Cybernetics.
5. Codecademy. “Breadth-First Search (BFS) Algorithm Explained.” *Codecademy*, 2025, www.codecademy.com/article/breadth-first-search-bfs-algorithm.
6. “Depth First Search Tutorials & Notes | Algorithms | HackerEarth.” *HackerEarth*, 2016, www.hackerearth.com/practice/algorithms/graphs/depth-first-search/tutorial/.
7. “IDA-Star (IDA*) Algorithm in General.” *Insight into Programming Algorithms*, 30 Mar. 2016, <https://algorithmsinsight.wordpress.com/graph-theory-2/ida-star-algorithm-in-general/>